AAI201 MACHINE LEARNING | CAPSTONE PROJECT

Predictive Maintenance Failure Prediction

Binary classification on the AI4I 2020 dataset using baseline and ensemble models

Drew Menning | DMACC, Spring 2026

github.com/drewmenning/ai4i-predictive-maintenance

Problem and dataset

Why predict machine failure?

Unplanned downtime is expensive and disruptive. A model that flags machines likely to fail in the near term lets a maintenance team intervene early. The cost of a missed failure is much higher than the cost of an unnecessary inspection, so recall and threshold choice matter more than raw accuracy.

Dataset: AI4I 2020 (UCI ML Repository)

Synthetic, tabular. Features include product type, air temperature, process temperature, rotational speed, torque, and tool wear. Target is the binary Machine failure flag.

10,000

rows in the dataset

3.39%

machines fail (339 of 10,000), a strongly imbalanced positive class

8

features used (after removing IDs and per-mode failure leak columns)

Approach: baseline to ensemble, with the right metric

Models compared

Logistic Regression

L2 regularized, balanced class weights

Decision Tree

max_depth = 6, min_samples_leaf = 20

Random Forest

300 trees, depth 12, balanced weights

HistGradientBoosting

Validation early stopping at 118 iters

Techniques

Class imbalance

class_weight='balanced' + threshold tuning (no SMOTE on synthetic data)

Threshold selection

F1-max on validation, applied unchanged to test

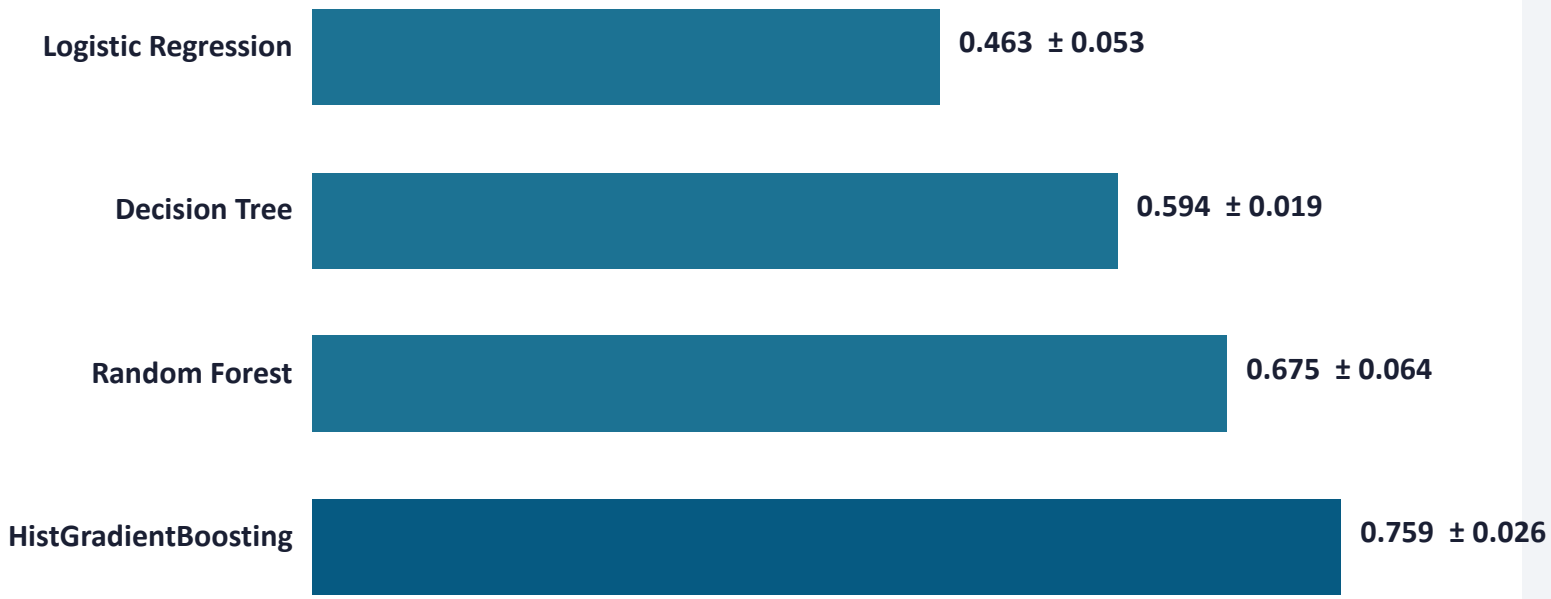
Overfitting controls

L2, depth limits, 5-fold stratified CV, early stopping

Fairness check

Per-Type recall/precision (L, M, H quality grades)

Model comparison: 5-fold cross-validation PR-AUC



PR-AUC (5-fold mean ± std on training set)

Why PR-AUC?

ROC-AUC stays high on imbalanced data because true negatives dominate. PR-AUC ignores those and rewards models that actually rank failures high.

HGB wins on mean and variance.

0.759 mean PR-AUC with the smallest spread (±0.026), so the choice is stable across folds, not lucky.

Final test results: HistGradientBoosting + tuned threshold

0.882
PR-AUC

0.804
Recall (failures caught)

0.911
Precision

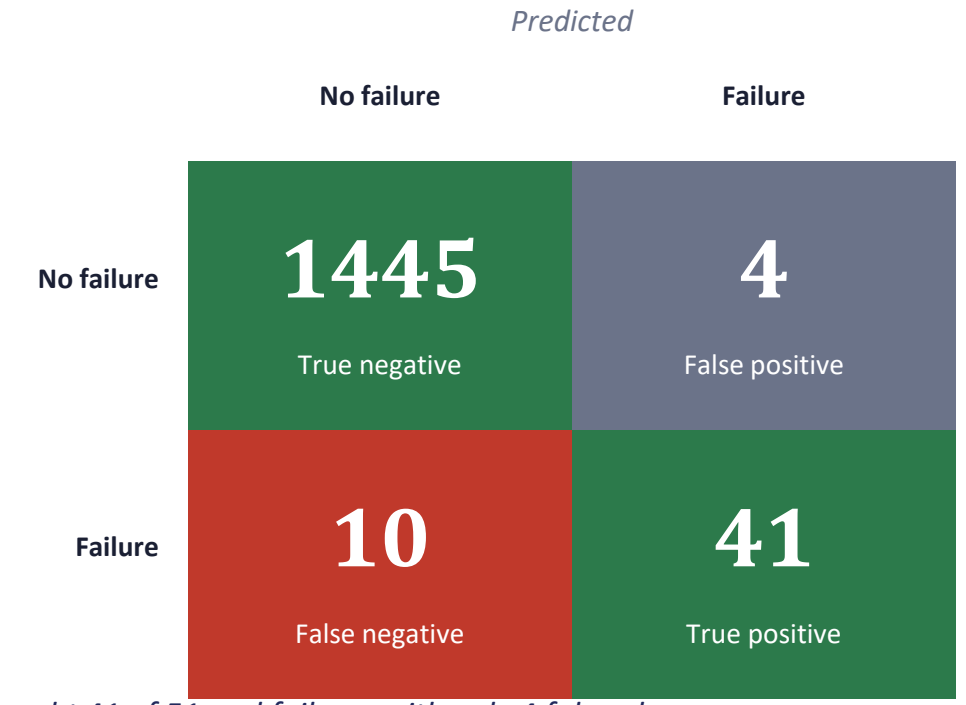
0.854
F1 score

0.981
ROC-AUC

0.991
Accuracy

Confusion matrix (held-out test set)

Threshold = 0.496, chosen by F1-max on validation



Caught 41 of 51 real failures with only 4 false alarms.

Real finding: the model fails on Type H machines

Validation-set performance by product quality type

Type	n	Failure rate	Recall	Precision
L	924	4.0%	0.649	0.96
M	459	2.2%	0.700	0.70
H	**118**	**3.4%**	**0.000**	**0.00**

On Type H machines, the model caught 0 of 4 failures.

Why this matters

H is the smallest grade in the dataset (only ~6%). With four real failures in validation, the model has almost nothing to learn from. The result generalizes: any deployment plan would need stratified sampling, a separate H-specific model, or an explicit out-of-scope flag for that grade.

Took the proposal's fairness check seriously and it surfaced a real problem, not a checkbox.

Reflection and next steps

What I learned

Threshold tuning and the right metric beat picking a fancier model. Decision Tree with PR-AUC 0.59 was not far behind a tuned booster at 0.76. Class imbalance is less about the model and more about how predictions are turned into decisions.

Hardest decision

Choosing the failure threshold. Default 0.5 produced near-zero recall on imbalanced data. F1-max on validation gave a defensible threshold (0.496) without needing a cost ratio I couldn't justify on synthetic data.

Honest limitation

AI4I 2020 is synthetic. Feature distributions and failure logic are generated, not measured. The model is a credible result on this dataset but not a guarantee on a real shop floor with sensor noise and timing offsets.

Next steps

1 **Real data**

Validate against NASA C-MAPSS or PHM Society datasets to test transfer.

2 **Cost-based threshold**

Replace F1-max with a threshold tied to documented FN vs FP costs.

3 **Type H repair**

Stratified resampling, an H-specific model, or an explicit out-of-scope flag.

4 **Drift monitoring**

Add input-feature drift checks and a recalibration trigger in production.

github.com/drewmenning/ai4i-predictive-maintenance